Defence Report

Z-Empire Auto Accessories Shop Management and Pricing System

Sean Esperancilla · Written in my own words, to read the night before and the morning of.

18	42	6	74	476	24	108	25
modules	tables	schemas	products	stored prices	controllers	views	defects

1. How I open, if they let me talk first

Z-Empire fits window tint, paint protection film and ceramic coating. Three kinds of work that are priced differently and stocked differently, all going through one counter.

Before this system, every quotation was read off a printed sheet and added up by hand. The same job could be quoted at two different prices by two members of staff on the same day, and nobody could check afterwards which was right. Film is cut from rolls, so nobody could say how much of a shade was left without walking to the rack. And the quotation, the job order, the receipt and the warranty were four separate pieces of paper with nothing connecting them.

What I built holds the shop's own rules in one place: a price book with four keys, stock measured in centimetres for film and counted in pieces for everything else, and one job carried from quotation through to warranty without re-typing.

Then stop. Let them ask.

2. The three ideas that are mine, not a tutorial's

Pricing has four keys	A price is **Product × Tint Variant × Vehicle Class × Panel**. Clear Series alone holds 134 stored prices across three variants, eight vehicle classes and six panels. The shade the customer picks does **not** change the price — it changes which roll the film is cut from. That one distinction is why there is no shade column in the pricing table, and it is the thing I would most expect to be questioned on.
Film is measured, not counted	A horn or a wax is one unit in, one unit out. Film leaves the shelf in centimetres, and how many depends on which panel of which size of car it was cut for. The cut-size table holds the shop's own measurements, one per vehicle class and panel, and completing a job order deducts exactly that.
Shade stock is derived, not stored	The stock ledger records a product and nothing finer — it knows three metres of film left the shelf, not whether it was Light, Medium or Superdark. So there is no running balance per shade to read. What I do instead: **left = what the last stock count found** − **what has gone on documents since**. The screen says it is an estimate and shows the ledger's own total beside it.

3. Methodology — the question I am most likely to be pressed on

> **Answer this one the same way every time**
> I used an **iterative waterfall** model. The six phases ran in order — Requirement Analysis, System Design, Implementation, Testing, Deployment, Maintenance — but testing and review regularly returned work to an earlier phase.

Q **Your paper says Waterfall. Is that really what you did?**

A Not the strict one-pass version, no, and I would rather say so than defend a diagram that does not match my own commit history. I measured it: **159 of 214 source files (74%) were modified again in a later week, 104 of them (49%) in three or more separate weeks, and only 55 (26%) were written once and never returned to.** Twenty-five documented defects each sent work back to an earlier phase. That is iterative waterfall, and it is what the figure in my paper shows.

Q **Is iterative waterfall a real model, or something you invented to fit?**

A It is a real one. Royce's 1970 paper, which the waterfall model is taken from, **included feedback between adjacent phases** — he presented the simple one-pass form as flawed. The strict linear version most textbooks draw is a later simplification of his paper.

Q **Why not Agile, then?**

A There was one developer and one client, and no sprint cadence or standups to speak of. Calling it Agile would have been a label rather than a description. What actually happened was the six phases, repeated, with feedback — so that is what I called it.

Q **How did you gather requirements?**

A From the shop itself: the printed price list on the wall, the way the staff quote a job aloud, and the paperwork they already kept. Most of the hard parts — four pricing keys, film measured in centimetres, shades that do not change the price — came out of watching how they already worked, not out of asking what features they wanted.

4. Architecture and database

Q **Why ASP.NET Core and SQL Server?**

A The shop runs on Windows, and SQL Server Express is free at this scale. ASP.NET Core 8 is cross-platform and supported for years, so the system is not tied to a runtime that stops being patched.

Q **Why are there no Entity Framework migrations?**

A The database existed first and holds the shop's live data. Generating it from code would mean regenerating something already in use. So the system is **database-first**: a schema change is made on the database and the model follows. The trade-off is that I cannot roll a schema change back as code, which is why I avoided schema changes entirely once the system was live.

Q **Why six schemas instead of one flat set of tables?**

A cat, crm, ops, inv, sales and sec — catalogue, customers, operations, inventory, sales, security. A table's purpose is readable before you open it, and it keeps 42 tables navigable. ASP.NET Identity keeps its own schema, asp, which is why the application maps 33 of the 42.

Q **What happens if two staff edit the same record at once?**

A Entity Framework raises a concurrency exception and the save is refused rather than one person silently overwriting the other. In a shop with a handful of staff it is rare, but it fails safely rather than quietly.

5. Inventory and the shade problem

Q **Why is shade stock an estimate rather than a fact?**

A Because inv.InventoryTransaction records a ProductID and nothing finer, so no individual shade has a running balance anywhere in the database. I could not add one without a schema change to a live database holding the shop's real data, and I was not prepared to make that mid-project. It is the first item on my recommendations as the single change that would fix it.

Q **Then how can the figure be trusted?**

A It is not presented as fact. The screen states it is worked out, shows the ledger's own total beside it as the authoritative figure, and names the two things that stop them agreeing: film issued on lines that record no shade, and deliveries received since the count, which are recorded against the product rather than a shade. A figure with its own error sources printed next to it is more useful than a confident wrong number.

Q **Is there a known inaccuracy?**

A Yes, and it is documented in the service itself. A document's **date** is treated as the moment its film left the shelf, because nothing records when a job was actually completed in a way I can join back to the stock movement. So a job raised before a count and completed after it gets subtracted twice. The error is **in the shop's favour** — it under-reports what is left, which sends someone to look at the rack rather than promising film that is not there — and it clears at the next count.

Q **How does the shop correct a wrong figure?**

A By counting. The Shade Stock page takes a count per shade directly, which writes a real stock-count record, and every figure is worked forward from the new baseline. There is nothing to edit because there is no stored number to edit.

6. Access control and security

Q **How do you stop a staff member doing an administrator's job?**

A Two levels. **Module access** decides which screens a role can open and is stored as data the owner edits. **Action rules** decide what can be done inside them and are enforced in code. Deleting is Admin-only across all **46 delete endpoints in 16 controllers**, applied by a single convention rather than 46 attributes — so an endpoint written next year is covered the moment it exists.

Q **How do you know the restrictions actually work?**

A I tested them by **typing the address directly** from a staff account, not by checking a button was missing. Hiding a control is presentation; the server-side check is the restriction. Both are in place, and the hiding exists so nobody reaches for something that will only refuse them.

Q **What about passwords?**

A ASP.NET Identity handles hashing. The development seed password is refused outright in Production, so the default that appears in public source cannot reach a live site. The database credential lives only on the server and is kept out of version control.

Q **Where are uploaded files kept, and can anyone read them?**

A Outside the web root, deliberately. Payment proofs carry bank reference numbers and checklist photos carry plate numbers; served as static files they would be readable by anyone who guessed the URL. They are streamed back through an action behind the module's own permission. The file name is built only from integers, so nothing a phone called a file reaches the disk, and the opening bytes are checked against the extension so a renamed executable is refused.

7. Testing

Q **How did you test it?**

A Four ways. **Functional** — each module through its own screens, a quotation taken through to a warranty claim on more than one class of vehicle. **Data audits** — scripts that read the catalogue and report what no screen can show. **Access testing** — every restriction tried by typing the address. **Deployment testing** — sign-in, PDF generation, session survival after an idle period, and a job order with a film line, run against the live site.

Q **Twenty-five defects sounds like a lot. Is that bad?**

A It is the number I found and wrote down, which is the point of writing them down. Two of the most serious looked completely correct on screen: renaming an employee updated the employee record but not their login, so the wrong name appeared on historical records; and setting a staff member inactive changed a badge in a list and nothing else — they could still sign in. Neither was visible without deliberately testing for it.

Q **Did you write automated tests?**

A Not a formal test suite, no. The verification I did was targeted at the parts where I did not trust my own reading — the file store's size and type checks, the panel de-duplication, the money formatting. For a system this size with one developer I would rather be honest that coverage is partial than claim a suite I do not have.

8. Deployment

Q **Is it actually deployed, or only running on your laptop?**

A Deployed, on live hosting, with the shop's real catalogue restored — 42 tables, 74 products, 476 prices. I can open it in front of you.

Q **What did you learn from deploying it?**

A Four faults that a development machine can never show. Shared hosting runs several sites in one worker, so the default in-process hosting model is refused and the site would not start at all. Sign-in keys held in memory are regenerated whenever the site idles, which logs everyone out mid-job. A redeploy would have overwritten the shop's own cut-size measurements. And the database credential had to come off the repository. Finding those was part of the work, not a step after it.

9. Where I am weakest — and what I say

> **The rule for all of these**
> Name it before they find it. A limitation I raise myself is a design decision; the same limitation found by a panel member is a hole. Every item below has an honest answer, and none of them is worth bluffing.

Q **The catalogue has gaps I know about.**

A My own audit script found them, which is the part worth saying. **BF Film has four variants and nine shades configured and zero prices**, so it cannot be quoted — the pricing script for it was never run against this database. **Carbon Tint has two prices and Premium Ceramic Tint has three**, against 134 for Clear Series, so they are barely configured. And forty paint-protection rows are priced against “Full Wrap”, which means all the glass, when they should be “Whole Vehicle”, which means the painted body. I found all of it by auditing my own data rather than by being told.

Q **Two panel names mean the same glass.**

A cat.Panel held “Front Windshield” beside “Front WS”. The price scripts create the short name; the long one was added by hand later. The checklist now collapses them so a car is not asked about its windscreen twice, and the rows it keeps are the ones the pricing is keyed on. The underlying duplicate rows are still there and I have a script that reports and merges them.

Q **Why are the panels named so confusingly?**

A They are, and that is my mistake rather than the shop's. “Full Wrap” and “Whole Vehicle” both read as the entire car to anyone who does not already know one means glass and the other means paint. It is on my recommendations to rename them, and I would do that before anything else on the list.

Q **There is no automated test suite.**

A Correct. I verified the parts I did not trust by eye and left the rest to functional testing. With one developer and a fixed deadline that was the trade I made, and I would spend the time on it next.

Q **The system is not in daily production use yet.**

A It is deployed and the shop's catalogue is in it. Whatever the current stage is, I will say it plainly rather than imply more.

10. The demo, in the order I will run it

1. **Sign in.** Done once before they walk in, so the first slow load of the day is not in front of them.
2. **A quotation** for a tint job — pick the product, the variant, the vehicle, and let the panel price fill itself in. This is the four-key pricing matrix doing its job, and it is the strongest thing I have.
3. **The vehicle checklist** — the walkaround panel by panel, with a photo attached. One row per piece of glass.
4. **The job order**, completed, and the stock coming off the roll in centimetres.
5. **The PDF receipt.** Riskiest piece: the PDF engine is a native 64-bit binary and the hosting app pool was set to 32-bit. **Test this before the defence.**
6. **Inventory** → **Shade stock** — and say out loud that it is derived, before anyone asks.
7. **Sign in as Staff** and type an admin-only address to show it refuses.

11. Tonight and tomorrow morning

- [ ] **Generate one PDF receipt on the live site.** The single most likely thing to fail in front of the panel.
- [ ] Sign in once so the site is warm.
- [ ] Have a job order with a film line, and a checklist with a photo, already in the data.
- [ ] Decide whether Chapter 3 says Waterfall or Iterative Waterfall — and make the figure, the prose and what I say out loud all agree. A figure that contradicts the paragraph beside it is worse than either alone.
- [ ] Read section 2 of this report once more. If I know only three things cold, they are those three.

If I am asked something I do not know: say so, say how I would find out, and move on. A panel forgives a gap far more readily than a guess that unravels halfway through.